

LAB #2

Q1. Create a new table Person and insert at least 5 records.

```
CREATE TABLE Person (PersonID INT PRIMARY KEY, Name VARCHAR(50) NOT NULL, Age INT, City VARCHAR(30), Phone VARCHAR(15));  
INSERT INTO Person VALUES (1, 'Ali', 25, 'Karachi', '0300-1111111'),  
(2, 'Sara', 30, 'Lahore', '0321-2222222'), (3, 'Usman', 22, 'Islamabad', '0333-3333333'),  
(4, 'Hina', 28, 'Quetta', '0345-4444444'), (5, 'Zaid', 35, 'Peshawar', '0311-5555555');
```

Q2. Create a new table Customer and insert at least 5 records.

```
CREATE TABLE Customer (CustomerID INT PRIMARY KEY, CustName VARCHAR(50) NOT NULL, City VARCHAR(30), Grade INT, Balance DECIMAL(10,2));  
  
INSERT INTO Customer VALUES (101, 'Ahmed', 'Karachi', 1, 5000.00),  
(102, 'Bilal', 'Lahore', 2, 3000.00), (103, 'Sana', 'Islamabad', 1, 7500.00),  
(104, 'Raza', 'Quetta', 3, 2000.00), (105, 'Nida', 'Peshawar', 2, 4500.00);
```

Q3. Create a new table Order and insert at least 5 records.

```
CREATE TABLE Orders (OrderID INT PRIMARY KEY, CustomerID INT, Amount DECIMAL(10,2), OrderDate DATE, FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID));  
  
INSERT INTO Orders VALUES (1001, 101, 1500.00, '2024-01-10'), (1002, 102, 800.00, '2024-01-12'),  
(1003, 103, 2200.00, '2024-01-15'), (1004, 104, 500.00, '2024-01-18'),  
(1005, 105, 1100.00, '2024-01-20');
```

Q4. Add a new field in the Person table.

```
ALTER TABLE Person ADD Email VARCHAR(100);
```

Q5. Modify any field in the Person table.

```
ALTER TABLE Person ALTER COLUMN Phone VARCHAR(20);
```

Q6. Use some constraints in your queries.

```
ALTER TABLE Person ADD CONSTRAINT chk_Age CHECK (Age >= 18 AND Age <= 100);  
  
ALTER TABLE Customer ADD CONSTRAINT uq_CustName UNIQUE (CustName);
```

LAB #3

Q1. Insert new employee's records.

```
INSERT INTO emp (empno, ename, job, sal, deptno) VALUES (8000, 'JOHN', 'CLERK', 1200, 30);
```

Q2. Insert new department — deptno=50, dname=ADVERTISING, loc=MIAMI.

```
INSERT INTO dept (deptno, dname, loc) VALUES (50, 'ADVERTISING', 'MIAMI');
```

Q3. Delete the records of Sales department.

```
DELETE FROM emp WHERE deptno = (SELECT deptno FROM dept WHERE dname = 'SALES');
```

Q4. Change the employee salary for SMITH.

```
UPDATE emp SET sal = 1500 WHERE ename = 'SMITH';
```

Q5. Confirm all changes by using SELECT statement.

```
SELECT * FROM emp; SELECT * FROM dept;
```

Q6. Update the salary of each employee to 5000.

```
UPDATE emp SET sal = 5000;
```

Q7. Change salary to \$1,000 for all employees with salary less than \$900.

```
UPDATE emp SET sal = 1000 WHERE sal < 900;
```

Q8. Change the commission of department 20 to 1000.

```
UPDATE emp SET comm = 1000 WHERE deptno = 20;
```

Q9. Change the hire date of all clerks to 02-04-2000.

```
UPDATE emp SET hiredate = '2000-04-02' WHERE job = 'CLERK';
```

Q10. Delete all records having hire date before 21-Dec-81.

```
DELETE FROM emp WHERE hiredate < '1981-12-21';
```

Q11. Delete all records where salary is greater than 2000.

```
DELETE FROM emp WHERE sal > 2000;
```

Q1. Insert a manager into emp.

```
INSERT INTO emp VALUES (9001, 'MARK', 'MANAGER', 7839, '2023-06-01', 3500, NULL, 10);
```

Q2. Insert a new analyst.

```
INSERT INTO emp (empno, ename, job, sal, deptno) VALUES (9002, 'LISA', 'ANALYST', 4000, 20);
```

Q3. Update job title of empno 7900.

```
UPDATE emp SET job = 'ANALYST' WHERE empno = 7900;
```

Q4. Give 10% raise to all clerks.

```
UPDATE emp SET sal = sal * 1.10 WHERE job = 'CLERK';
```

Q5. Set commission to 500 for all salesmen.

```
UPDATE emp SET comm = 500 WHERE job = 'SALESMAN';
```

Q6. Transfer all dept 30 employees to dept 40.

```
UPDATE emp SET deptno = 40 WHERE deptno = 30;
```

Q7. Delete employee with empno 7369.

```
DELETE FROM emp WHERE empno = 7369;
```

Q8. Delete all employees with no commission.

```
DELETE FROM emp WHERE comm IS NULL;
```

Q9. Insert a record into dept.

```
INSERT INTO dept VALUES (60, 'RESEARCH', 'BOSTON');
```

Q10. Update location of dept 10 to NEW YORK.

```
UPDATE dept SET loc = 'NEW YORK' WHERE deptno = 10;
```

LAB #4

Q1. List all rows of the table emp.

```
SELECT * FROM emp;
```

Q2. List all rows of dept.

```
SELECT * FROM dept;
```

Q3. List all employees' number from emp.

```
SELECT empno FROM emp;
```

Q4. List all employees' name from emp.

```
SELECT ename FROM emp;
```

Q5. List all departments' number from emp.

```
SELECT DISTINCT deptno FROM emp;
```

Q6. Find all employees whose salaries are between 500 and 1500.

```
SELECT * FROM emp WHERE sal BETWEEN 500 AND 1500;
```

Q7. Find all employees whose salaries are between 1500 and 2500.

```
SELECT * FROM emp WHERE sal BETWEEN 1500 AND 2500;
```

Q8. Find all employees whose salaries are between 2600 and 5000.

```
SELECT * FROM emp WHERE sal BETWEEN 2600 AND 5000;
```

Q9. Find all employees whose salaries are less than 2000.

```
SELECT * FROM emp WHERE sal < 2000;
```

Q10. Find all employees whose salaries are greater than 2000.

```
SELECT * FROM emp WHERE sal > 2000;
```

Q11. Find employees whose mgr are 7902, 7566, 7788.

```
SELECT * FROM emp WHERE mgr IN (7902, 7566, 7788);
```

Q12. Find employees whose mgr is between 7788 and 7902.

```
SELECT * FROM emp WHERE mgr BETWEEN 7788 AND 7902;
```

Q13. List all employees whose name starts with 's'.

```
SELECT * FROM emp WHERE ename LIKE 'S%';
```

Q14. List all employees whose name starts with 'a'.

```
SELECT * FROM emp WHERE ename LIKE 'A%';
```

Q15. List all employees having first name as 'SCOTT'.

```
SELECT * FROM emp WHERE ename = 'SCOTT';
```

Q16. List all employees having first name as 'SMITH' or 'KING'.

```
SELECT * FROM emp WHERE ename IN ('SMITH','KING');
```

Q17. List employees with first letter 'W' or 'K' not in dept 10, 20, 40.

```
SELECT * FROM emp WHERE ename LIKE 'W%' OR ename LIKE 'K%' AND deptno NOT IN (10,20,40);
```

Q18. List employees whose name begins with 'SCOTT' and ends with 'MARTIN'.

```
SELECT * FROM emp WHERE ename LIKE 'SCOTT%' OR ename LIKE '%MARTIN';
```

Q19. List employees whose name starts with 'm' and belong to department 30.

```
SELECT * FROM emp WHERE ename LIKE 'M%' AND deptno = 30;
```

Q20. List employees whose name starts with 'b' and their job is manager.

```
SELECT * FROM emp WHERE ename LIKE 'B%' AND job = 'MANAGER';
```

Q21. List all employees who do not have job manager.

```
SELECT * FROM emp WHERE job != 'MANAGER';
```

Q22. Find employees whose job does not start with 'a'.

```
SELECT * FROM emp WHERE job NOT LIKE 'A%';
```

Q23. Find employees whose job does not start with 'c'.

```
SELECT * FROM emp WHERE job NOT LIKE 'C%';
```

Q24. List employees whose mgr is not null.

```
SELECT * FROM emp WHERE mgr IS NOT NULL;
```

Q25. List employees whose jobs are 'MANAGER' or 'CLERK' in dept 10.

```
SELECT * FROM emp WHERE job IN ('MANAGER','CLERK') AND deptno = 10;
```

Q26. List employees whose jobs are 'ANALYST' and 'SALESMAN' in dept 30.

```
SELECT * FROM emp WHERE job IN ('ANALYST','SALESMAN') AND deptno = 30;
```

Q27. Find all clerks who earn salaries between 1000 and 2000.

```
SELECT * FROM emp WHERE job = 'CLERK' AND sal BETWEEN 1000 AND 2000;
```

Q28. Find all managers who earn salaries between 2500 and 3000.

```
SELECT * FROM emp WHERE job = 'MANAGER' AND sal BETWEEN 2500 AND 3000;
```

Q29. Find all CLERK or MANAGER with salaries in range 1000 to 2000.

```
SELECT * FROM emp WHERE job IN ('CLERK','MANAGER') AND sal BETWEEN 1000 AND 2000;
```

Q30. Find all MANAGER and/or employees with salaries between 2000 and 3000.

```
SELECT * FROM emp WHERE job = 'MANAGER' OR sal BETWEEN 2000 AND 3000;
```

Q31. Find employees with salary = 1500 and job MANAGER or SALESMAN.

```
SELECT * FROM emp WHERE sal = 1500 AND job IN ('MANAGER','SALESMAN');
```

LAB #5

Q1. Find the join of relations EMPLOYEE and EQUIPMENT.

```
SELECT E.*, Q.* FROM EMPLOYEE E INNER JOIN EQUIPMENT Q ON E.EMP_ID = Q.EMP_ID;
```

Q2. Get all employees for projects using EQP-TYPE as a 'Welding machine'.

```
SELECT E.* FROM EMPLOYEE E INNER JOIN EQUIPMENT Q ON E.EMP_ID = Q.EMP_ID WHERE Q.EQP_TYPE = 'Welding machine';
```

Q3. Get all machines being used at the Mumbai Project.

```
SELECT Q.* FROM EQUIPMENT Q WHERE Q.PROJECT = 'Mumbai';
```

Q4. Find all employees of the project using equipment number 110.

```
SELECT E.* FROM EMPLOYEE E INNER JOIN EQUIPMENT Q ON E.EMP_ID = Q.EMP_ID WHERE Q.EQP_ID = 110;
```

Q5. Which salesman are working for which customer?

```
SELECT S.NAME AS Salesman, C.CUST_NAME AS Customer FROM SALESMAN S INNER JOIN CUSTOMER C ON S.SALESMAN_ID = C.SALESMAN_ID;
```

Q6. List in ascending order customers who work through a salesman or by own.

```
SELECT C.CUST_NAME, S.NAME FROM CUSTOMER C LEFT JOIN SALESMAN S ON C.SALESMAN_ID = S.SALESMAN_ID ORDER BY C.CUST_NAME ASC;
```

Q7. List salesmen who work for one or more customers or not yet joined any.

```
SELECT S.NAME, C.CUST_NAME FROM SALESMAN S LEFT JOIN CUSTOMER CONS ON S.SALESMAN_ID = C.SALESMAN_ID ORDER BY S.NAME ASC;
```

Q8. Report with customer name, city, order no, order date, purchase amount (placed orders or not in list).

```
SELECT C.CUST_NAME, C.CITY, O.ORD_NO, O.ORD_DATE, O.PURCH_AMT FROM CUSTOMER C FULL OUTER JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID ORDER BY C.CUST_NAME;
```

LAB #6

Q1. Group the employees by their salaries.

```
SELECT sal, COUNT(*) AS emp_count FROM emp GROUP BY sal;
```

Q2. List hire dates in descending order.

```
SELECT ename, hiredate FROM emp ORDER BY hiredate DESC;
```

Q3. List hire dates in ascending order.

```
SELECT ename, hiredate FROM emp ORDER BY hiredate ASC;
```

Q4. List all employees and add 20 rupees in each salary.

```
SELECT ename, sal + 20 AS new_sal FROM emp;
```

Q5. List all 'MANAGER' and add 100 rupees in each salary.

```
SELECT ename, sal + 100 AS new_sal FROM emp WHERE job = 'MANAGER';
```

Q6. List all 'SALESMAN' and add 500 rupees in each salary.

```
SELECT ename, sal + 500 AS new_sal FROM emp WHERE job = 'SALESMAN';
```

Q7. List all 'CLERK' and add 50 rupees in each salary.

```
SELECT ename, sal + 50 AS new_sal FROM emp WHERE job = 'CLERK';
```

Q8. Find eight percent of salesmen salary.

```
SELECT ename, sal * 0.08 AS eight_percent FROM emp WHERE job = 'SALESMAN';
```

Q9. Find the annual salary of each employee.

```
SELECT ename, sal * 12 AS annual_sal FROM emp;
```

Q10. Find the six-month salary of each employee.

```
SELECT ename, sal * 6 AS six_month_sal FROM emp;
```

Q11. Find the two-month salary of each manager.

```
SELECT ename, sal * 2 AS two_month_sal FROM emp WHERE job = 'MANAGER';
```

Q12. Make a query where all arithmetic expressions result remains the same.

```
SELECT ename, sal, sal + 0 AS same_sal, sal * 1 AS same_sal2 FROM emp;
```

Q13. Find the daily wages of each employee.

```
SELECT ename, ROUND(sal / 30, 2) AS daily_wage FROM emp;
```

Q14. Find the daily wages of each 'MANAGER'.

```
SELECT ename, ROUND(sal / 30, 2) AS daily_wage FROM emp WHERE job = 'MANAGER';
```

Q15. Display your name in lowercase.

```
SELECT LOWER(ename) AS lower_name FROM emp;
```

Q16. Find the first and second characters of enames.

```
SELECT SUBSTRING(ename, 1, 1) AS first_char, SUBSTRING(ename, 2, 1) AS second_char FROM emp;
```

Q17. Find the minimum salary from emp.

```
SELECT MIN(sal) AS min_salary FROM emp;
```

Q18. Find the maximum salary from emp.

```
SELECT MAX(sal) AS max_salary FROM emp;
```

Q19. Find the length of all ename.

```
SELECT ename, LEN(ename) AS name_length FROM emp;
```

Q20. Find the length of job.

```
SELECT job, LEN(job) AS job_length FROM emp;
```

Q21. Find the sum of all salaries.

```
SELECT SUM(sal) AS total_salary FROM emp;
```

Q22. Find employees whose department location is Newark.

```
SELECT e.* FROM emp e JOIN dept d ON e.deptno = d.deptno WHERE d.loc = 'NEWARK';
```

Q23. Find employees working in accounting department.

```
SELECT e.* FROM emp e JOIN dept d ON e.deptno = d.deptno WHERE d.dname = 'ACCOUNTING';
```

Q24. Count all employees.

```
SELECT COUNT(*) AS total_employees FROM emp;
```

Q25. Display the sum of all employees' salaries.

```
SELECT SUM(sal) AS total_sal FROM emp;
```

Q26. How many managers do we have?

```
SELECT COUNT(*) AS manager_count FROM emp WHERE job = 'MANAGER';
```

Q27. How many departments do we have?

```
SELECT COUNT(*) AS dept_count FROM dept;
```

Q28. List average salary of each job.

```
SELECT job, AVG(sal) AS avg_salary FROM emp GROUP BY job;
```

Q29. Find the Maximum and Minimum salary of all employees.

```
SELECT MAX(sal) AS max_sal, MIN(sal) AS min_sal FROM emp;
```

Q30. Find the average salaries of those employees who work in dept 10.

```
SELECT AVG(sal) AS avg_sal FROM emp WHERE deptno = 10;
```

Q31. Find average and sum of all salaries of each job excluding clerks.

```
SELECT job, AVG(sal) AS avg_sal, SUM(sal) AS sum_sal FROM emp WHERE job != 'CLERK' GROUP BY job;
```

Q32. Find the minimum and average salary of each department excluding deptno 10.

```
SELECT deptno, MIN(sal) AS min_sal, AVG(sal) AS avg_sal FROM emp WHERE deptno != 10 GROUP BY deptno;
```

Lab #7

Q1. One-to-One: Each employee has one passport.

```
CREATE TABLE Passport (PassportID INT PRIMARY KEY, EmpNo INT UNIQUE, PassportNo VARCHAR(20), FOREIGN KEY (EmpNo) REFERENCES emp(empno));
```

Q2. One-to-Many: One department has many employees (already in emp/dept schema).

```
SELECT d.dname, COUNT(e.empno) AS emp_count FROM dept d LEFT JOIN emp e ON d.deptno = e.deptno GROUP BY d.dname;
```

Q3. Many-to-Many: Employees work on multiple projects (junction table).

```
CREATE TABLE EMP_PROJECT (EmpNo INT, ProjID INT, Role VARCHAR(30), PRIMARY KEY (EmpNo, ProjID), FOREIGN KEY (EmpNo) REFERENCES emp(empno));
```

Q4. Many-to-One: Many orders belong to one customer.

```
SELECT o.ORD_NO, c.CUST_NAME FROM ORDERS o INNER JOIN CUSTOMER c ON o.CUSTOMER_ID = c.CUSTOMER_ID;
```

LAB #8

Q1. Display employee name and hiredate for all in same dept as Blake; exclude Blake.

```
SELECT ename, hiredate FROM emp WHERE deptno = (SELECT deptno FROM emp WHERE ename = 'BLAKE') AND ename != 'BLAKE';
```

Q2. Display employee number and name for all who earn more than average salary.

```
SELECT empno, ename FROM emp WHERE sal > (SELECT AVG(sal) FROM emp);
```

Q3. Display empno and name for employees who work in a dept with any employee whose name contains a 'T'.

```
SELECT empno, ename FROM emp WHERE deptno IN (SELECT deptno FROM emp WHERE ename LIKE '%T%');
```

Q4. Display name, deptno, and job for all employees whose department location is Dallas.

```
SELECT e.ename, e.deptno, e.job FROM emp e JOIN dept d ON e.deptno = d.deptno WHERE d.loc = 'DALLAS';
```

Q5. Display employee name and salary of all who report to KING.

```
SELECT ename, sal FROM emp WHERE mgr = (SELECT empno FROM emp WHERE ename = 'KING');
```

Q6. Display name, salary, deptno and job for employees in same job as empno 7369.

```
SELECT ename, sal, deptno, job FROM emp WHERE job = (SELECT job FROM emp WHERE empno = 7369);
```

Q7. Display empno, name, salary for those who earn more than average and work in a dept with a 'T' in any employee name.

```
SELECT empno, ename, sal FROM emp WHERE sal > (SELECT AVG(sal) FROM emp) AND deptno IN (SELECT deptno FROM emp WHERE ename LIKE '%T%');
```

Lab #9

Q1. Display employee name, department name, and location for employees who earn a commission.

```
SELECT e.ename, d.dname, d.loc FROM emp e JOIN dept d ON e.deptno = d.deptno WHERE e.comm IS NOT NULL;
```

Q2. Display all employees' name (including KING who has no manager) and their manager name.

```
SELECT w.ename AS Employee, m.ename AS Manager FROM emp w LEFT JOIN emp m ON w.mgr = m.empno;
```

Q3. Display the name of all employees whose manager is KING.

```
SELECT ename FROM emp WHERE mgr = (SELECT empno FROM emp WHERE ename = 'KING');
```

Q4. Create a unique listing of all jobs in department 30 including location.

```
SELECT DISTINCT e.job, d.loc FROM emp e JOIN dept d ON e.deptno = d.deptno WHERE e.deptno = 30;
```

Q5. Display name, job, department number and department name for employees in New York.

```
SELECT e.ename, e.job, e.deptno, d.dname FROM emp e JOIN dept d ON e.deptno = d.deptno WHERE d.loc = 'NEW YORK';
```

Q6. Display employee name & number along with their manager's name and number.

```
SELECT w.empno AS Emp_Num, w.ename AS Employee, m.empno AS Mgr_Num, m.ename AS Manager FROM emp w LEFT JOIN emp m ON w.mgr = m.empno;
```

LAB #10

Q1. CREATE SEQUENCE my_sequence MINVALUE 1 MAXVALUE 1000 START WITH 1 INCREMENT BY 2.

```
CREATE SEQUENCE my_sequence MINVALUE 1 MAXVALUE 1000 START WITH 1 INCREMENT BY 2;
```

Q2. Create a new table Person with my_sequence and insert at least 5 records.

```
CREATE TABLE Person (PersonID INT DEFAULT NEXT VALUE FOR my_sequence PRIMARY KEY, Name VARCHAR(50), City VARCHAR(30));
```

```
INSERT INTO Person (Name, City) VALUES ('Ali','Karachi'), ('Sara','Lahore'), ('Usman','Islamabad'), ('Hina','Quetta'), ('Zaid','Peshawar');
```

Q3. Create a new table Customer with my_sequence and insert at least 5 records.

```
CREATE TABLE Customer (CustID INT DEFAULT NEXT VALUE FOR my_sequence PRIMARY KEY, CustName VARCHAR(50), City VARCHAR(30));
```

```
INSERT INTO Customer (CustName, City) VALUES ('Ahmed','Karachi'), ('Bilal','Lahore'), ('Sana','Islamabad'), ('Raza','Quetta'), ('Nida','Peshawar');
```

Q4. Create B-Tree indexes: i) Name column of EMP ii) Designation column of EMP iii) First 10 chars of Title in TRAINING.

```
CREATE INDEX idx_emp_name ON emp(ename);
```

```
CREATE INDEX idx_emp_desig ON emp(job);
```

```
CREATE INDEX idx_training_title10 ON TRAINING(SUBSTR(Title, 1, 10));
```

Q5. Create bitmap indexes: i) Gender column of EMP ii) Performance column of EMP_PROJECT.

```
CREATE BITMAP INDEX idx_emp_gender ON emp(Gender);
```

```
CREATE BITMAP INDEX idx_empproj_perf ON EMP_PROJECT(Performance);
```

LAB #11

Q1. Create EMPLOYEE table.

```
CREATE TABLE EMPLOYEE (Empno INT PRIMARY KEY, Name VARCHAR(50) NOT NULL, Designation VARCHAR(50), Qualification VARCHAR(30), Joindate DATE);
```

Q2. Create PROJECT table.

```
CREATE TABLE PROJECT (PID INT PRIMARY KEY, Title VARCHAR(100), Client VARCHAR(50), Duration INT, Status VARCHAR(20), CONSTRAINT chk_status CHECK (Status IN ('New','In Progress','Complete')));
```

Q3. Create EMP_PROJECT junction table.

```
CREATE TABLE EMP_PROJECT (Empno INT, PID INT, Performance VARCHAR(10), PRIMARY KEY (Empno, PID), CONSTRAINT chk_perf CHECK (Performance IN ('Excellent','Good','Fair','Bad','Poor')), FOREIGN KEY (Empno) REFERENCES EMPLOYEE(Empno), FOREIGN KEY (PID) REFERENCES PROJECT(PID));
```

Q4. Create GRADE table.

```
CREATE TABLE GRADE (Designation VARCHAR(50) PRIMARY KEY, Grade INT CHECK (Grade BETWEEN 1 AND 20), TotalPosts INT, PostsAvailable INT, CONSTRAINT chk_posts CHECK (PostsAvailable <= TotalPosts));
```

Q5. Create TRAINING table.

```
CREATE TABLE TRAINING (Tcode INT PRIMARY KEY, Title VARCHAR(100), StartDate DATE, EndDate DATE);
```

Q6. Create EMP_TRAINING junction table.

```
CREATE TABLE EMP_TRAINING (Empno INT, Tcode INT, Attendance DECIMAL(5,2), PRIMARY KEY (Empno, Tcode), FOREIGN KEY (Empno) REFERENCES EMPLOYEE(Empno), FOREIGN KEY (Tcode)
```

```
REFERENCES TRAINING(Tcode));
```

Q7. Add Gender column to EMP table (Male/Female only).

```
ALTER TABLE EMPLOYEE ADD Gender VARCHAR(10); ALTER TABLE EMPLOYEE ADD CONSTRAINT  
chk_gender CHECK (Gender IN ('Male','Female'));
```

Q8. Add Instructor_Name column to TRAINING table.

```
ALTER TABLE TRAINING ADD Instructor_Name VARCHAR(50);
```

Q9. Add Salary column to GRADE table.

```
ALTER TABLE GRADE ADD Salary DECIMAL(10,2);
```

Q10. Insert Employee 3400 with Developer 6i training (87% attendance) — commit transaction.

```
BEGIN TRANSACTION; INSERT INTO EMP_TRAINING VALUES (3400, (SELECT Tcode FROM TRAINING  
WHERE Title='Developer 6i'), 87); COMMIT;
```

Q11. Insert Employee 3300 with Typing/Shorthand training — commit transaction.

```
BEGIN TRANSACTION; INSERT INTO EMP_TRAINING VALUES (3300, (SELECT Tcode FROM TRAINING  
WHERE Title='Typing/Shorthand'), 75); COMMIT;
```

Q12. Create a view showing employees with their project performance.

```
CREATEVIEWvw_EmpPerformanceASSELECTe.Name,e.Designation,ep.PerformanceFROM EMPLOYEE e  
JOIN EMP_PROJECT ep ON e.Empno = ep.Empno;
```

Q13. Create a view showing products above average price (Brazil Customers example).

```
CREATE VIEW BrazilCustomers AS SELECT CustomerName, ContactName, City FROM Customers WHERE  
Country = 'Brazil';
```

Q14. Update the view to add City column.

```
CREATEORREPLACEVIEWBrazilCustomersASSELECTCustomerName,ContactName,CityFROM Customers  
WHERE Country = 'Brazil';
```

Q15. Drop the Brazil Customers view.

```
DROP VIEW BrazilCustomers;
```

LAB #12

Q1. Create the ACCOUNTING_USER profile with given resource and password limits.

```
CREATEPROFILEACCOUNTING_USERLIMITSESSIONS_PER_USER6CONNECT_TIME1440IDLE_TIME30  
FAILED_LOGIN_ATTEMPTS 4 PASSWORD_LOCK_TIME UNLIMITED PASSWORD_LIFE_TIME 60  
PASSWORD_GRACE_TIME 5 PASSWORD_REUSE_TIME 90 PASSWORD_REUSE_MAX UNLIMITED;
```

Q2. Assign the ACCOUNTING_USER profile to user SCOTT.

```
ALTER USER SCOTT PROFILE ACCOUNTING_USER;
```

Q3. Drop the ACCOUNTING_USER profile.

```
DROP PROFILE ACCOUNTING_USER CASCADE;
```

Q4. Create user AHMED with profile, tablespace and quota.

```
CREATE USER AHMED IDENTIFIED BY GREEN DEFAULT TABLESPACE USERS QUOTA 2M ON USERS PROFILE  
ACCOUNTING_USER ACCOUNT LOCK;
```

Q5. Unlock user AHMED's account.

```
ALTER USER AHMED ACCOUNT UNLOCK;
```

Q6. Expire user AHMED's password.

```
ALTER USER AHMED PASSWORD EXPIRE;
```

Q7. Drop user AHMED with all owned objects.

```
DROP USER AHMED CASCADE;
```

Q8. Grant SELECT and UPDATE on CUSTOMER table to YASIR.

```
GRANT SELECT, UPDATE ON CUSTOMER TO YASIR;
```

Q9. Grant INSERT (specific columns) and DELETE on CUSTOMER to YASIR with grant option.

```
GRANT INSERT(Customer_id, Name), DELETE ON CUSTOMER TO YASIR WITH GRANT OPTION;
```

Q10. Grant system privileges to AHMED with admin option.

```
GRANT CREATE SESSION, CREATE TABLE, CREATE INDEX TO AHMED WITH ADMIN OPTION;
```

Q11. Grant any table and sequence access to AHMED.

```
GRANT CREATE ANY TABLE, SELECT ANY TABLE, SELECT ANY SEQUENCE TO AHMED;
```

Q12. Revoke UPDATE privilege from YASIR on CUSTOMER.

```
REVOKE UPDATE ON CUSTOMER FROM YASIR;
```

Q13. Create a MANAGER role and grant it CREATE TABLE and CREATE VIEW.

```
CREATE ROLE manager; GRANT CREATE TABLE, CREATE VIEW TO manager;
```

Q14. Grant manager role to users AHMED and YASIR.

```
GRANT manager TO AHMED, YASIR;
```

Q15. Create tablespace ACCOUNT_DATA with two data files.

```
CREATETABLESPACEACCOUNT_DATAFILE'd:\oradata\db01\acct_data01.dbf'SIZE100M AUTOEXTEND  
ON NEXT 5M MAXSIZE 200M, 'd:\oradata\db01\acct_data02.dbf' SIZE 100M;
```

Q16. Enable resource limits on the database.

```
ALTER SYSTEM SET RESOURCE_LIMIT = TRUE;
```

LAB #13

Q1. Creating a server audit with a file target (HIPAA_Audit).

```
CREATE SERVER AUDIT HIPAA_Audit TO FILE (FILEPATH = '\\SQLPROD_1\Audit\');  
ALTER SERVER AUDIT HIPAA_Audit WITH (STATE = ON);
```

Q2. Creating a server audit with a Windows Application Log target with options.

```
CREATE SERVER AUDIT HIPAA_Audit TO APPLICATION_LOG WITH (QUEUE_DELAY = 1000, ON_FAILURE =  
SHUTDOWN);
```

```
ALTER SERVER AUDIT HIPAA_Audit WITH (STATE = ON);
```

Q3. Creating a server audit containing a WHERE clause (filter on SensitiveData table only).

```
CREATE SERVER AUDIT AuditDataAccess TO FILE (FILEPATH = 'C:\SQLAudit\') WHERE object_name  
= 'SensitiveData';
```

```
ALTER SERVER AUDIT AuditDataAccess WITH (STATE = ON);
```

```
CREATE DATABASE AUDIT SPECIFICATION [FilterForSensitiveData] FOR SERVER AUDIT  
[AuditDataAccess] ADD (SELECT ON SCHEMA::[DataSchema] BY [public]) WITH (STATE = ON);
```